pid 16 fork 刚完成：proc / 内核栈的内核地址、用户地址空间，以及它们落在哪些物理页

上：虚拟地址视角（init 刚 fork 出、将 exec sh）。下：物理内存 0x6ead000–0x6ed3fff，每列一页

pid 16 · name=init（将 exec /bin/sh）

struct proc（内核虚拟地址，直接映射区）

0xffffff8006ed3000 = KERNBASE + pa 0x6ed3000

pid=16 tgid=16 state=1 USED
kstack = 0xffffffbfffe2000 （槽 14）
trapframe = 0xffffffbfffe2ef0
vm = 0xffffff8006ed1000

p->context @ 0xffffff8006ed30b0 (proc + 0xb0)
sp = 0xffffffbfffe2ef0 ← 现在与 trapframe 同址
x30 = forkret x18–x29 = 0
只有 swtch 会改写它（保存当时的内核 sp）

0xffffffbfffe3000 … +0x1000: 未映射（保护）

内核栈（KSTACK 槽 14，pa 0x6ed2000）

top = 0xffffffbfffe3000

trapframe（272 B）= top − 0x110：每次 el0sync 都在这里重写
elr = 0xd5bc libc __syscall+0x20
spsr = 0x60000000 （Z C · EL0t · 中断开）
sp = 0x89d0 用户栈指针（SP_EL0）
x0 = 0 子进程的 fork 返回值
x16/x17 = 0x133e0 / 0xd59c GOT / __syscall
x27 = 0x13650 libc 数据段结尾

0xffffffbfffe2ef0 = p->trapframe = context.sp（fork 时）

进入系统调用后：usertrap → syscall → sys_xxx → sleep → sched
的调用帧依次压在 trapframe 下面（向低地址）

sched() 调 swtch 时：context.sp ← 此刻的 sp（示意位置）

下次被调度：swtch 恢复它，从 sched 返回，原路回到 usertrap

base = 0xffffffbfffe2000

用户地址空间（TTBR0） sz = 0x14000 · 19 个用户页

TTBR0 页表根 pa 0x6eb8000
L1[0] → L2 页表 pa 0x6eb5000
L2[0] → L3 页表 pa 0x6ec0000
叶子 flags 0x34b：用户可读写、可执行

VA 0x0
- **程序 /init** 0x0000–0x2000
- **ld-xv6.so** 0x2000–0x4000
- **保护页（PTE_U 已清）** 0x4000–0x5000
- **用户栈 4 页** 0x5000–0x9000 — **user sp 0x89d0 → pa 0x6ece9d0**
- **elr 0xd5bc → pa 0x6eaf5bc**
- **libc 代码** 0x9000–0x122ba
- **libc 数据 + 堆** 0x132c0–0x14000

VA 0x14000

内核虚拟地址（TTBR1）

MAXVA = 0xffffffc000000000（高地址）

KSTACK(slot) = MAXVA − (slot+1)·0x2000：经页表映射

0xffffffbfffe3000 未映射（保护）
0xffffffbfffe2000 slot 14 内核栈 → pa 0x6ed2000
…

直接映射区：VA = KERNBASE + PA

0xffffff8006ed3000 pid 16 struct proc
0xffffff8006ed1000 pid 16 vmspace
0xffffff8006eb8000 页表根 L1
0xffffff8006eb5000 L2 页表
0xffffff8006ec0000 L3 页表

KERNBASE = 0xffffff8000000000（低地址）

context.sp 什么时候变

fork 后（本图）	context.sp = trapframe 地址 …ef0
第一次调度	swtch 载入它 → forkret → usertrapret → eret 回到 EL0，SP_EL1 回到栈顶
每次 svc / 中断	el0sync：SP_EL1 = 栈顶，sub sp, #272 → trapframe 总在同一位置被重写 context.sp 不变（只有 swtch 写它）
系统调用里睡眠	sleep → sched → swtch(&p->context, …) context.sp ← 当时的 sp，低于 trapframe
被唤醒再调度	swtch 恢复 → 从 sched 返回 → syscall → usertrapret 用 trapframe 回到 EL0
被时钟抢占	el0irq → yield → sched：同样更新

0x6ead000 VA 0xb000 libc 代码
0x6eae000 VA 0xc000 libc 代码
0x6eaf000 VA 0xd000 libc 代码
0x6eb0000 VA 0xe000 libc 代码
0x6eb1000 VA 0xf000 libc 代码
0x6eb2000 VA 0x10000 libc 代码
0x6eb3000 VA 0x11000 libc 代码
0x6eb4000 VA 0x12000 libc 代码
0x6eb5000 L2 页表
0x6eb6000 VA 0x0 程序
0x6eb7000 VA 0x13000 libc 数据/堆
0x6eb8000 页表根 L1
0x6eb9000 VA 0xa000 libc 代码
0x6eba000 VA 0x6000 栈
0x6ebb000 VA 0x5000 栈
0x6ebc000 VA 0x4000 保护页?
0x6ebd000 VA 0x3000 ld-xv6
0x6ebe000 VA 0x2000 ld-xv6
0x6ebf000 VA 0x1000 程序
0x6ec0000 L3 页表
0x6ec1000
0x6ec2000
0x6ec3000
0x6ec4000
0x6ec5000
0x6ec6000
0x6ec7000
0x6ec8000
0x6ec9000
0x6eca000
0x6ecb000
0x6ecc000
0x6ecd000
0x6ece000 VA 0x7000 栈
0x6ecf000 VA 0x8000 栈
0x6ed0000
0x6ed1000 VA 0x9000 libc 代码
0x6ed2000 vmspace
0x6ed3000 内核栈 struct proc

↑ 物理内存（每列一页，地址从左到右递增）

- 内核对象：直接映射区 PA = VA − 0xffffff8000000000（struct proc、vmspace、页表）；内核栈在 KSTACK 槽，经页表映射 0xffffffbfffe2000 → 0x6ed2000。
- p->context 在 struct proc 内部（+0xb0），箭头落在 proc 那一页；trapframe 与 fork 时的 context.sp 落在内核栈那一页 +0xef0 处。
- 用户地址查 L3 叶子再加页内偏移：sp 0x89d0 → L3[8] 0x6ece000 + 0x9d0；elr 0xd5bc → L3[13] 0x6eaf000 + 0x5bc。
- 空白格 = 快照里没有这一页（空闲或属于别的进程）；“?” = 保护页，按页号空缺推测（已分配，但清掉了 PTE_U，dump 不打印）。